

Министерство науки и высшего образования РФ
ФГАОУ ВПО
Национальный исследовательский технологический университет
«МИСИС»

Институт информационных технологий и компьютерных наук (ИТКН)

Кафедра инфокоммуникационных технологий (ИКТ)

Отчет по лабораторной работе № 11

по дисциплине «Объектно-ориентированное программирование»
на тему «Работа с протоколом http в C#. Асинхронное программирование в C#.»

Выполнил:
студент группы БИВТ-25-1
Кирюшин А. А.
Проверил:
доцент каф. ИКТ
Стучилин В.В.

Москва, 2026

1. ЦЕЛЬ И ЗАДАЧИ РАБОТЫ

Цель работы: познакомиться с основами взаимодействия с REST API в C#: изучить принципы работы протокола HTTP, форматы передачи данных JSON, механизм авторизации через токен, научиться выполнять CRUD-операции через HTTP-запросы.

Задачи:

1. Реализовать авторизацию с помощью HTTP-запроса POST /auth и получение токена.
2. Реализовать получение списка идентификаторов бронирований с помощью GET /booking.
3. Реализовать получение детальной информации о бронировании по идентификатору с помощью GET /booking/{id}.
4. Использовать асинхронные методы async/await, чтобы сетевые запросы не блокировали интерфейс приложения.
5. Использовать библиотеку Newtonsoft.Json для сериализации и десериализации JSON-данных.
6. Выполнить дополнительное задание: реализовать полный набор CRUD-операций над бронированиями с использованием методов POST, PUT и DELETE.

Вариант работы: разработано настольное приложение-клиент для сервиса бронирований. Пользователь может авторизоваться, загрузить список бронирований, просмотреть данные в таблице, создать новое бронирование, изменить выбранное бронирование и удалить его.

2. ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В лабораторной работе использованы следующие технологии:

1. Язык программирования C# и платформа .NET.
2. Avalonia UI для построения кроссплатформенного графического интерфейса.
3. Паттерн MVVM и библиотека CommunityToolkit.Mvvm для организации логики представления.
4. System.Net.Http.HttpClient для выполнения HTTP-запросов.

5. Newtonsoft.Json для преобразования объектов C# в JSON и обратно.
6. MessageBox.Avalonia для отображения ошибок и диалогов подтверждения.

3. ОСНОВНЫЕ КОМПОНЕНТЫ И ИСХОДНЫЙ КОД

3.1 Модели данных

Файл BookingModels.cs содержит классы, соответствующие структурам JSON, которые использует REST API. Отдельно описаны тело запроса авторизации, ответ с токеном, идентификатор бронирования, даты бронирования и основная сущность Booking.

Класс BookingWithId добавлен для удобного отображения данных в DataGridView: он хранит идентификатор бронирования и предоставляет вычисляемые свойства для колонок таблицы.

Модели данных (BookingModels.cs)

Полный код: Models/BookingModels.cs

```
using System;
```

```
namespace HotelBookingApp.Models;
```

```
public class AuthRequest
```

```
{  
    public string username { get; set; } = string.Empty;  
    public string password { get; set; } = string.Empty;  
}
```

```
public class AuthResponse
```

```
{  
    public string token { get; set; } = string.Empty;  
}
```

```
public class BookingIdResponse
```

```
{  
    public int bookingid { get; set; }  
}
```

```
public class BookingDates
```

```
{
    public string checkin { get; set; } = string.Empty;
    public string checkout { get; set; } = string.Empty;
}
```

public class Booking

```
{
    public string firstname { get; set; } = string.Empty;
    public string lastname { get; set; } = string.Empty;
    public int totalprice { get; set; }
    public bool depositpaid { get; set; }
    public BookingDates bookingdates { get; set; } = new BookingDates();
    public string additionalneeds { get; set; } = string.Empty;
}
```

public class BookingWithId

```
{
    public int Id { get; set; }
    public Booking BookingDetails { get; set; } = new Booking();

    // Свойства для удобного биндинга в DataGridView
    public string FirstName => BookingDetails.firstname;
    public string LastName => BookingDetails.lastname;
    public string TotalPrice => $" {BookingDetails.totalprice} руб.";
    public string DepositPaid => BookingDetails.depositpaid ? "Да" : "Нет";
    public string CheckIn => BookingDetails.bookingdates.checkin;
    public string CheckOut => BookingDetails.bookingdates.checkout;
    public string AdditionalNeeds => BookingDetails.additionalneeds;
}
```

3.2 Сервис работы с REST API

Класс `BookingApiService` инкапсулирует все сетевые операции. В нем используется один статический экземпляр `HttpClient`, что соответствует рекомендуемой практике и предотвращает лишнее создание сетевых ресурсов.

Сервис выполняет следующие операции:

1. `AuthenticateAsync` отправляет POST `/auth`, получает токен и сохраняет его для последующих защищенных запросов.

2. GetBookingIdsAsync получает список идентификаторов бронирований.
3. GetBookingDetailsAsync загружает подробные данные по конкретному ID.
4. CreateBookingAsync создает бронирование методом POST /booking.
5. UpdateBookingAsync обновляет выбранное бронирование методом PUT /booking/{id} и передает токен в cookie.
6. DeleteBookingAsync удаляет выбранное бронирование методом DELETE /booking/{id}.

Сетевой сервис (BookingApiService.cs)

Полный код: Services/BookingApiService.cs

```
using System;
using System.Collections.Generic;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Text;
using System.Threading.Tasks;
using HotelBookingApp.Models;
using Newtonsoft.Json;

namespace HotelBookingApp.Services;

public class BookingApiService
{
    private const string BASE_URL = "http://restful-booker.herokuapp.com";

    // Один статический экземпляр HttpClient на всё приложение (best practice)
    private static readonly HttpClient Http = new HttpClient
    {
        Timeout = TimeSpan.FromSeconds(15)
    };

    private string _token = string.Empty;
    public bool IsAuthenticated => !string.IsNullOrEmpty(_token);

    public BookingApiService()
    {
```

// Устанавливаем глобальный заголовок Accept, чтобы всегда получать JSON

```
if (!Http.DefaultRequestHeaders.Accept.Contains(new MediaTypeWithQualityHeaderValue("application/json")))
{
    Http.DefaultRequestHeaders.Accept.Add(new MediaTypeWithQualityHeaderValue("application/json"));
}
```

/// <summary>

/// Авторизация и получение токена

/// </summary>

```
public async Task<string> AuthenticateAsync(string username, string password)
{
    var reqBody = new AuthRequest { username = username, password = password };
    string jsonBody = JsonConvert.SerializeObject(reqBody);
    var content = new StringContent(jsonBody, Encoding.UTF8, "application/json");

    var response = await Http.PostAsync($"{BASE_URL}/auth", content);
    response.EnsureSuccessStatusCode();

    string responseStr = await response.Content.ReadAsStringAsync();
    var authObj = JsonConvert.DeserializeObject<AuthResponse>(responseStr);

    if (authObj == null || string.IsNullOrEmpty(authObj.token) || authObj.token == "Bad credentials")
    {
        throw new Exception("Неверный логин или пароль");
    }

    _token = authObj.token;
    return _token;
}
```

/// <summary>

/// Получение списка всех ID бронирований

/// </summary>

```
public async Task<List<int>> GetBookingIdsAsync()
{
    var response = await Http.GetAsync($"{BASE_URL}/booking");
    response.EnsureSuccessStatusCode();

    string json = await response.Content.ReadAsStringAsync();
    var ids = JsonConvert.DeserializeObject<List<BookingIdResponse>>(json);

    var result = new List<int>();
    if (ids != null)
    {
        foreach (var item in ids)
        {
            result.Add(item.bookingid);
        }
    }
    return result;
}
```

/// <summary>

/// Получение деталей конкретного бронирования

/// </summary>

```
public async Task<BookingWithId?> GetBookingDetailsAsync(int id)
{
    var response = await Http.GetAsync($"{BASE_URL}/booking/{id}");
    if (!response.IsSuccessStatusCode)
        return null;

    string json = await response.Content.ReadAsStringAsync();
    // Проверка на случай, если сервер вернул XML или HTML (например, 418
I'm a teapot)
    if (json.TrimStart().StartsWith("<"))
        return null;

    var booking = JsonConvert.DeserializeObject<Booking>(json);
    if (booking == null) return null;
}
```

```

    return new BookingWithId { Id = id, BookingDetails = booking };
}

/// <summary>
/// Создание нового бронирования (POST) - не требует токена
/// </summary>
public async Task<BookingWithId> CreateBookingAsync(Booking booking)
{
    string jsonBody = JsonConvert.SerializeObject(booking);
    var content = new StringContent(jsonBody, Encoding.UTF8, "application/json");

    // Для POST /booking также нужно добавить Accept заголовок явно на всякий случай
    var req = new HttpRequestMessage(HttpMethod.Post, $"{BASE_URL}/booking")
    {
        Content = content
    };
    req.Headers.Accept.Add(new MediaTypeWithQualityHeaderValue("application/json"));

    var response = await Http.SendAsync(req);
    response.EnsureSuccessStatusCode();

    string responseStr = await response.Content.ReadAsStringAsync();

    // Ответ при создании содержит и ID, и само бронирование
    // Формат: {"bookingid": 123, "booking": { ... }}
    var createdObj = JsonConvert.DeserializeObject<dynamic>(responseStr);
    int newId = createdObj!.bookingid;

    return new BookingWithId { Id = newId, BookingDetails = booking };
}

/// <summary>
/// Обновление бронирования (PUT) - ТРЕБУЕТ ТОКЕН
/// </summary>
public async Task UpdateBookingAsync(int id, Booking booking)

```



```

{
    if (!IsAuthenticated) throw new Exception("Необходима авторизация");

    string jsonBody = JsonConvert.SerializeObject(booking);

    var req = new HttpRequestMessage(HttpMethod.Put, $"{BASE_URL}/booking/{id}");
    req.Content = new StringContent(jsonBody, Encoding.UTF8, "application/json");
    req.Headers.Accept.Add(new MediaTypeWithQualityHeaderValue("application/json"));
    req.Headers.Add("Cookie", $"token={_token}");

    var response = await Http.SendAsync(req);
    response.EnsureSuccessStatusCode();
}

/// <summary>
/// Удаление бронирования (DELETE) - ТРЕБУЕТ TOKEN
/// </summary>
public async Task DeleteBookingAsync(int id)
{
    if (!IsAuthenticated) throw new Exception("Необходима авторизация");

    var req = new HttpRequestMessage(HttpMethod.Delete, $"{BASE_URL}/booking/{id}");
    req.Headers.Add("Cookie", $"token={_token}");

    var response = await Http.SendAsync(req);
    // Сервер restful-booker возвращает 201 Created при успешном удалении
    if (!response.IsSuccessStatusCode)
    {
        throw new Exception($"Ошибка удаления. Код: {response.StatusCode}");
    }
}
}

```

3.3 ViewModel главного окна

MainWindowViewModel связывает интерфейс и сетевой сервис. В нем реализованы команды авторизации, загрузки данных, добавления, редактирования и удаления бронирований. Все операции, связанные с сетью, выполняются асинхронно, поэтому интерфейс остается отзывчивым.

При загрузке данных приложение сначала получает список ID, затем батчами запрашивает подробности бронирований через Task.WhenAll. Это позволяет выполнять несколько HTTP-запросов параллельно и быстрее получить первые успешные записи.

Логика главного окна (MainWindowViewModel.cs)

Полный код: ViewModels/MainWindowViewModel.cs

```
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Threading.Tasks;
using Avalonia.Threading;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using HotelBookingApp.Models;
using HotelBookingApp.Services;

namespace HotelBookingApp.ViewModels;

public partial class MainWindowViewModel : ObservableObject
{
    private readonly BookingApiService _api = new();

    [ObservableProperty]
    private string _username = "admin";

    [ObservableProperty]
    private string _password = "password123";

    [ObservableProperty]
    private string _statusMessage = "Ожидание авторизации...";

    [ObservableProperty]
```

```
[NotifyPropertyChanged(nameof(CanLoad))]
```

```
[NotifyPropertyChanged(nameof(CanAdd))]
```

```
private bool _isAuthenticated = false;
```

```
[ObservableProperty]
```

```
[NotifyPropertyChanged(nameof(CanLoad))]
```

```
[NotifyPropertyChanged(nameof(CanAdd))]
```

```
[NotifyPropertyChanged(nameof(CanEditOrDelete))]
```

```
private bool _isLoading = false;
```

```
public bool CanLoad => IsAuthenticated && !isLoading;
```

```
public bool CanAdd => IsAuthenticated && !isLoading;
```

```
[ObservableProperty]
```

```
[NotifyPropertyChanged(nameof(CanEditOrDelete))]
```

```
private BookingWithId? _selectedBooking;
```

```
public bool CanEditOrDelete => SelectedBooking != null && !isLoading;
```

```
public ObservableCollection<BookingWithId> Bookings { get; } = new();
```

```
// События для вызова диалоговых окон из View
```

```
public Func<Booking?, Task<Booking?>>? ShowBookingDialogAsync;
```

```
public Func<string, Task<bool>>? ShowConfirmDialogAsync;
```

```
public Action<string>? ShowErrorAsync;
```

```
[RelayCommand]
```

```
private async Task LoginAsync()
```

```
{
```

```
    try
```

```
    {
```

```
        isLoading = true;
```

```
        StatusMessage = "Авторизация...";
```

```
        string token = await _api.AuthenticateAsync(Username, Password);
```

```
        IsAuthenticated = true;
```

```
        StatusMessage = $"Авторизован. Токен: {token}";
```

```
    }
```

```

catch (Exception ex)
{
    StatusMessage = $"Ошибка: {ex.Message}";
    ShowErrorAsync?.Invoke(ex.Message);
}
finally
{
    IsLoading = false;
}
}

```

[RelayCommand]

```

private async Task LoadBookingsAsync()
{
    try
    {
        IsLoading = true;
        Bookings.Clear();
        StatusMessage = "Получение списка ID...";

        var ids = await _api.GetBookingIdsAsync();
        StatusMessage = $"Найдено {ids.Count} ID. Загружаю детали (батчами)
...";

        int loadedCount = 0;
        int targetCount = 10; // По заданию нужно 10 успешных
        int batchSize = 20;

        for (int i = 0; i < ids.Count && loadedCount < targetCount; i += batchSize)
        {
            var tasks = new List<Task<BookingWithId?>>();
            int currentBatchSize = Math.Min(batchSize, ids.Count - i);

            for (int j = 0; j < currentBatchSize; j++)
            {
                tasks.Add(_api.GetBookingDetailsAsync(ids[i + j]));
            }

            // Ждем завершения всех запросов в батче параллельно

```

```

var results = await Task.WhenAll(tasks);

foreach (var result in results)
{
    if (result != null && loadedCount < targetCount)
    {
        Bookings.Add(result);
        loadedCount++;
    }
}

StatusMessage = $"Готово. Показано {loadedCount} бронирований.";
}
catch (Exception ex)
{
    StatusMessage = $"Ошибка загрузки: {ex.Message}";
}
finally
{
    IsLoading = false;
}
}

```

```

[RelayCommand]
private async Task AddBookingAsync()
{
    if (ShowBookingDialogAsync == null) return;

    var newBookingData = await ShowBookingDialogAsync(null);
    if (newBookingData == null) return; // Отмена

    try
    {
        IsLoading = true;
        StatusMessage = "Создание бронирования...";

        var created = await _api.CreateBookingAsync(newBookingData);
    }
}

```

```

        // Добавляем в начало списка
        Bookings.Insert(0, created);
        SelectedBooking = created;
        StatusMessage = $"Бронирование #{created.Id} успешно создано.";
    }
    catch (Exception ex)
    {
        StatusMessage = "Ошибка создания";
        ShowErrorAsync?.Invoke(ex.Message);
    }
    finally
    {
        IsLoading = false;
    }
}

```

[RelayCommand]

private async Task EditBookingAsync()

```

{
    if (SelectedBooking == null || ShowBookingDialogAsync == null) return;

    var currentData = SelectedBooking.BookingDetails;
    var updatedData = await ShowBookingDialogAsync(currentData);
    if (updatedData == null) return; // Отмена

    try
    {
        IsLoading = true;
        StatusMessage = $"Обновление бронирования #{SelectedBooking.Id}..."
    ;

```

```

        await _api.UpdateBookingAsync(SelectedBooking.Id, updatedData);

```

// Создаем новый объект, чтобы Avalonia DataGrid гарантированно заметил изменения и обновил строку

```

    int idx = Bookings.IndexOf(SelectedBooking);
    if (idx >= 0)
    {
        var updatedItem = new BookingWithId { Id = SelectedBooking.Id, Book

```

```

ingDetails = updatedData };
    Bookings[idx] = updatedItem;
    SelectedBooking = updatedItem;
}

    StatusMessage = $"Бронирование #{SelectedBooking.Id} успешно обнов
лено.";
}
catch (Exception ex)
{
    StatusMessage = "Ошибка обновления";
    ShowErrorAsync?.Invoke(ex.Message);
}
finally
{
    IsLoading = false;
}
}

```

```

[RelayCommand]
private async Task DeleteBookingAsync()
{
    if (SelectedBooking == null || ShowConfirmDialogAsync == null) return;

    bool confirm = await ShowConfirmDialogAsync($"Вы действительно хотит
е удалить бронирование #{SelectedBooking.Id} ({SelectedBooking.FirstName}
{SelectedBooking.LastName})?");
    if (!confirm) return;

    try
    {
        IsLoading = true;
        StatusMessage = $"Удаление бронирования #{SelectedBooking.Id}...";

        await _api.DeleteBookingAsync(SelectedBooking.Id);

        Bookings.Remove(SelectedBooking);
        SelectedBooking = null;
    }
}

```

```

        StatusMessage = "Бронирование успешно удалено.";
    }
    catch (Exception ex)
    {
        StatusMessage = "Ошибка удаления";
        ShowErrorAsync?.Invoke(ex.Message);
    }
    finally
    {
        IsLoading = false;
    }
}
}

```

3.4 Главное окно приложения

Главное окно состоит из трех областей: верхней панели авторизации и команд CRUD, центральной таблицы бронирований и нижней строки статуса. Таблица выводит ID, имя, фамилию, цену, признак оплаты депозита, даты заезда и выезда, а также дополнительные пожелания.

Разметка главного окна (MainWindow.axaml)

Полный код: Views/MainWindow.axaml

```

<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:vm="using:HotelBookingApp.ViewModels"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        mc:Ignorable="d" d:DesignWidth="800" d:DesignHeight="600"
        x:Class="HotelBookingApp.Views.MainWindow"
        x:DataType="vm:MainWindowViewModel"
        Title="Лабораторная работа №11 - REST API Клиент"
        Width="900" Height="600">

    <Design.DataContext>
        <vm:MainWindowViewModel/>
    </Design.DataContext>

    <Grid RowDefinitions="Auto, *, Auto" Margin="10">
        <!-- Верхняя панель: Авторизация и Управление -->

```



```
<Border Grid.Row="0" BorderBrush="LightGray" BorderThickness="1" Padding="10" Margin="0,0,0,10" CornerRadius="5">
```

```
<Grid ColumnDefinitions="Auto, Auto, Auto, Auto, *, Auto, Auto, Auto, Auto">
```

```
<TextBlock Text="Логин:" VerticalAlignment="Center" Margin="0,0,5,0"/>
```

```
<TextBox Grid.Column="1" Text="{Binding Username}" Width="100" Margin="0,0,15,0" IsEnabled="{Binding !IsAuthenticated}"/>
```

```
<TextBlock Grid.Column="2" Text="Пароль:" VerticalAlignment="Center" Margin="0,0,5,0"/>
```

```
<TextBox Grid.Column="3" Text="{Binding Password}" PasswordChar="*" Width="100" Margin="0,0,15,0" IsEnabled="{Binding !IsAuthenticated}"/>
```

```
<Button Grid.Column="4" Content="Войти" Command="{Binding LoginCommand}" IsEnabled="{Binding !IsAuthenticated}" HorizontalAlignment="Left"/>
```

```
<!-- Кнопки CRUD -->
```

```
<Button Grid.Column="5" Content="Загрузить (10 шт)" Command="{Binding LoadBookingsCommand}" IsEnabled="{Binding CanLoad}" Margin="0,0,10,0"/>
```

```
<Button Grid.Column="6" Content="Добавить" Command="{Binding AddBookingCommand}" IsEnabled="{Binding CanAdd}" Margin="0,0,10,0"/>
```

```
<Button Grid.Column="7" Content="Редактировать" Command="{Binding EditBookingCommand}" IsEnabled="{Binding CanEditOrDelete}" Margin="0,0,10,0"/>
```

```
<Button Grid.Column="8" Content="Удалить" Command="{Binding DeleteBookingCommand}" IsEnabled="{Binding CanEditOrDelete}" Background="#FFE57373"/>
```

```
</Grid>
```

```
</Border>
```

```
<!-- Центральная часть: Таблица данных -->
```

```
<DataGrid Grid.Row="1"
```

```
ItemsSource="{Binding Bookings}"
```

```
SelectedItem="{Binding SelectedBooking}"
```

```
AutoGenerateColumns="False"
```

```
IsReadOnly="True"
```

```

        GridLinesVisibility="All"
        BorderThickness="1" BorderBrush="Gray">
<DataGrid.Columns>
    <DataGridTextColumn Header="ID" Binding="{Binding Id}" />
    <DataGridTextColumn Header="Имя" Binding="{Binding FirstName
}" />
    <DataGridTextColumn Header="Фамилия" Binding="{Binding LastName}" />
    <DataGridTextColumn Header="Цена" Binding="{Binding TotalPrice
}" />
    <DataGridTextColumn Header="Депозит" Binding="{Binding DepositPaid}" />
    <DataGridTextColumn Header="Заезд" Binding="{Binding CheckIn}"
" />
    <DataGridTextColumn Header="Выезд" Binding="{Binding CheckOut}" />
    <DataGridTextColumn Header="Пожелания" Binding="{Binding AdditionalNeeds}" Width="*" />
</DataGrid.Columns>
</DataGrid>

<!-- Нужная панель: Статус -->
<Border Grid.Row="2" Background="#FFF0F0F0" BorderBrush="LightGray" BorderThickness="1" Padding="5" Margin="0,10,0,0" CornerRadius="3">
    <TextBlock Text="{Binding StatusMessage}" FontWeight="SemiBold"/>
</Border>
</Grid>
</Window>

```

Файл code-behind подключает ViewModel и задает функции открытия диалогов. Такой подход оставляет сетевую и бизнес-логику во ViewModel, а работу с конкретными окнами выполняет представление.

Code-behind главного окна (MainWindow.axaml.cs)

Полный код: Views/MainWindow.axaml.cs

```

using System.Threading.Tasks;
using Avalonia.Controls;
using HotelBookingApp.Models;
using HotelBookingApp.ViewModels;

```

```

using MsBox.Avalonia;
using MsBox.Avalonia.Enums;

namespace HotelBookingApp.Views;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();

        var vm = new MainWindowViewModel();
        DataContext = vm;

        // Подключаем диалоги
        vm.ShowErrorAsync = async (msg) =>
        {
            var box = MessageBoxManager.GetMessageBoxStandard("Ошибка", msg,
                ButtonEnum.Ok, MsBox.Avalonia.Enums.Icon.Error);
            await box.ShowAsync();
        };

        vm.ShowConfirmDialogAsync = async (msg) =>
        {
            var box = MessageBoxManager.GetMessageBoxStandard("Подтверждение", msg, ButtonEnum.YesNo, MsBox.Avalonia.Enums.Icon.Question);
            var result = await box.ShowAsync();
            return result == ButtonResult.Yes;
        };

        vm.ShowBookingDialogAsync = async (booking) =>
        {
            var dialog = new BookingDialog(booking);
            var result = await dialog.ShowDialog<Booking?>(this);
            return result;
        };
    }
}

```

3.5 Диалог создания и редактирования бронирования

Окно BookingDialog используется и для добавления, и для редактирования бронирования. Если в конструктор передано существующее бронирование, поля формы заполняются его значениями. Если бронирование создается с нуля, даты заезда и выезда устанавливаются по умолчанию.

Разметка диалога бронирования (BookingDialog.axaml)

Полный код: Views/BookingDialog.axaml

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        mc:Ignorable="d" d:DesignWidth="400" d:DesignHeight="450"
        x:Class="HotelBookingApp.Views.BookingDialog"
        Title="Бронирование"
        Width="400" Height="450"
        WindowStartupLocation="CenterOwner">

    <Grid RowDefinitions="*, Auto" Margin="20">
        <StackPanel Grid.Row="0" Spacing="10">
            <TextBlock Text="Имя:" />
            <TextBox Name="FirstNameBox" />

            <TextBlock Text="Фамилия:" />
            <TextBox Name="LastNameBox" />

            <TextBlock Text="Цена:" />
            <TextBox Name="PriceBox" />

            <CheckBox Name="DepositBox" Content="Депозит оплачен" />

            <TextBlock Text="Дата заезда (YYYY-MM-DD):" />
            <TextBox Name="CheckInBox" />

            <TextBlock Text="Дата выезда (YYYY-MM-DD):" />
            <TextBox Name="CheckOutBox" />

            <TextBlock Text="Дополнительные пожелания:" />
            <TextBox Name="NeedsBox" />
        </StackPanel>
    </Grid>
</Window>
```

```

</StackPanel>

<StackPanel Grid.Row="1" Orientation="Horizontal" HorizontalAlignment=
"Right" Spacing="10" Margin="0,20,0,0">
    <Button Content="Отмена" Click="Cancel_Click" />
    <Button Content="Сохранить" Click="Save_Click" Background="#FF4C
AF50" Foreground="White" />
</StackPanel>
</Grid>
</Window>

```

Логика диалога бронирования (BookingDialog.axaml.cs)

Полный код: Views/BookingDialog.axaml.cs

```

using System;
using Avalonia.Controls;
using Avalonia.Interactivity;
using HotelBookingApp.Models;

namespace HotelBookingApp.Views;

public partial class BookingDialog : Window
{
    public BookingDialog()
    {
        InitializeComponent();
    }

    public BookingDialog(Booking? existingBooking) : this()
    {
        if (existingBooking != null)
        {
            FirstNameBox.Text = existingBooking.firstname;
            LastNameBox.Text = existingBooking.lastname;
            PriceBox.Text = existingBooking.totalprice.ToString();
            DepositBox.IsChecked = existingBooking.depositpaid;
            CheckInBox.Text = existingBooking.bookingdates.checkin;
            CheckOutBox.Text = existingBooking.bookingdates.checkout;
            NeedsBox.Text = existingBooking.additionalneeds;
        }
    }
}

```

```

else
{
    // Дефолтные значения для нового
    CheckInBox.Text = DateTime.Now.ToString("yyyy-MM-dd");
    CheckOutBox.Text = DateTime.Now.AddDays(1).ToString("yyyy-MM-dd
");
}
}

```

```

private void Save_Click(object? sender, RoutedEventArgs e)
{
    // Базовая валидация
    if (string.IsNullOrEmpty(FirstNameBox.Text) ||
        string.IsNullOrEmpty(LastNameBox.Text) ||
        !int.TryParse(PriceBox.Text, out int price))
    {
        // В реальном приложении тут нужно показать ошибку валидации
        return;
    }

```

```

var booking = new Booking
{
    firstname = FirstNameBox.Text,
    lastname = LastNameBox.Text,
    totalprice = price,
    depositpaid = DepositBox.IsChecked ?? false,
    bookingdates = new BookingDates
    {
        checkin = CheckInBox.Text ?? "",
        checkout = CheckOutBox.Text ?? ""
    },
    additionalneeds = NeedsBox.Text ?? ""
};

```

```

    Close(booking);
}

```

```

private void Cancel_Click(object? sender, RoutedEventArgs e)
{

```

```

        Close(null);
    }
}

```

3.6 Файл проекта

В файле проекта подключены пакеты Avalonia, DataGrid, CommunityToolkit.Mvvm, MessageBox.Avalonia и Newtonsoft.Json.

Файл проекта (HotelBookingApp.csproj)

Полный код: HotelBookingApp.csproj

```

<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ApplicationManifest>app.manifest</ApplicationManifest>
    <AvaloniaUseCompiledBindingsByDefault>true</AvaloniaUseCompiledBin
dingsByDefault>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Avalonia" Version="11.0.10" />
    <PackageReference Include="Avalonia.Controls.DataGrid" Version="11.0.10"
  />
    <PackageReference Include="Avalonia.Desktop" Version="11.0.10" />
    <PackageReference Include="Avalonia.Diagnostics" Version="11.0.10" />
    <PackageReference Include="Avalonia.Fonts.Inter" Version="11.0.10" />
    <PackageReference Include="Avalonia.Themes.Fluent" Version="11.0.10" />
    <PackageReference Include="CommunityToolkit.Mvvm" Version="8.4.2" />
    <PackageReference Include="MessageBox.Avalonia" Version="3.1.5.1" />
    <PackageReference Include="Newtonsoft.Json" Version="13.0.4" />
  </ItemGroup>
</Project>

```

4. СКРИНШОТЫ РАБОТЫ ПРИЛОЖЕНИЯ

Ниже представлены скриншоты, демонстрирующие работу приложения.

Рис. 1 — Авторизация в приложении и получение токена.

Лабораторная работа №11 - REST API Клиент

Логин: admin

Пароль: *****

Войти

Загрузить (10 шт)

Добавить

Редактировать

Удалить

ID	Имя	Фамилия	Цена	Депозит	Заезд	Выезд	Пожелания
----	-----	---------	------	---------	-------	-------	-----------

Ожидание авторизации...

Рис. 2 — Загрузка списка бронирований из REST API.

Лабораторная работа №11 - REST API Клиент

Логин: admin

Пароль: *****

Войти

Загрузить (10 шт)

Добавить

Редактировать

Удалить

ID	Имя	Фамилия	Цена	Депозит	Заезд	Выезд	Пожелания
2	Eric	Smith	498 руб.	Да	2025-12-26	2026-01-29	Breakfast
3	Mark	Ericsson	569 руб.	Нет	2018-12-12	2021-11-22	
4	Mary	Wilson	624 руб.	Да	2016-09-03	2023-02-06	Breakfast
5	Jim	Brown	292 руб.	Нет	2020-01-26	2022-09-01	Breakfast
6	Sally	Jackson	737 руб.	Да	2025-11-01	2025-12-10	Breakfast
7	Mark	Ericsson	991 руб.	Нет	2023-04-10	2026-01-24	Breakfast
8	Eric	Brown	896 руб.	Да	2020-12-16	2026-03-28	Breakfast
9	Jim	Jackson	148 руб.	Нет	2020-09-21	2024-11-06	Breakfast
10	Eric	Smith	101 руб.	Нет	2024-01-23	2024-06-06	
11	Jim	Brown	111 руб.	Да	2025-02-01	2025-02-05	Lunch

Готово. Показано 10 бронирований.

Рис. 3 — Создание нового бронирования через диалоговое окно.

admin

Пароль:

ID	Имя	Фамилия
2	Eric	Smith
3	Mark	Ericsson
4	Mary	Wilson
5	Jim	Brown
6	Sally	Jackson
7	Mark	Ericsson
8	Eric	Brown
9	Jim	Jackson
10	Eric	Smith
11	Jim	Brown

Имя:

aleksey

Фамилия:

kiryushin

Цена:

10000

☒ Депозит оплачен

Дата заезда (YYYY-MM-DD):

2026-05-17

Дата выезда (YYYY-MM-DD):

2026-05-18

Дополнительные пожелания:

-

Отмена

Сохранить

Редактировать

Удалить

Пожелания

Breakfast

Breakfast

Breakfast

Breakfast

Breakfast

Breakfast

Breakfast

Lunch

Готово. Показано 10 бронирований.

Рис. 4 — Добавление новой записи в таблицу.

Лабораторная работа №11 - REST API Клиент

admin

Пароль:

Войти

Загрузить (10 шт)

Добавить

Редактировать

Удалить

ID	Имя	Фамилия	Цена	Депозит	Заезд	Выезд	Пожелания
3555	aleksey	kiryushin	10000 руб.	Да	2026-05-17	2026-05-18	-
2	Eric	Smith	498 руб.	Да	2025-12-26	2026-01-29	Breakfast
3	Mark	Ericsson	569 руб.	Нет	2018-12-12	2021-11-22	
4	Mary	Wilson	624 руб.	Да	2016-09-03	2023-02-06	Breakfast
5	Jim	Brown	292 руб.	Нет	2020-01-26	2022-09-01	Breakfast
6	Sally	Jackson	737 руб.	Да	2025-11-01	2025-12-10	Breakfast
7	Mark	Ericsson	991 руб.	Нет	2023-04-10	2026-01-24	Breakfast
8	Eric	Brown	896 руб.	Да	2020-12-16	2026-03-28	Breakfast
9	Jim	Jackson	148 руб.	Нет	2020-09-21	2024-11-06	Breakfast
10	Eric	Smith	101 руб.	Нет	2024-01-23	2024-06-06	
11	Jim	Brown	111 руб.	Да	2025-02-01	2025-02-05	Lunch

Бронирование #3555 успешно создано.

Рис. 5 — Подтверждение удаления выбранного бронирования.

Лабораторная работа №11 - REST API Клиент

Логин: admin

Пароль: *****

Войти

Загрузить (10 шт)

Добавить

Редактировать

Удалить

ID	Имя	Фамилия	Цена	Депозит	Заезд	Выезд	Пожелания
3555	aleksey	kiryushin	10000 руб.	Да	2026-05-17	2026-05-18	-
2	Eric	Smith	498 руб.	Да	2025-12-26	2026-01-29	Breakfast
3	Mark	Ericsson	569 руб.	Нет	2018-12-12	2021-11-22	
4	Mary	Wilson	624 руб.	Да	2016-09-03	2023-02-06	Breakfast
5	Jim	Brown	292 руб.	Нет	2020-01-26	2022-09-01	Breakfast
6	Sally	Jackson	737 руб.	Да	2025-11-01	2025-12-10	Breakfast
7	Mark	Ericsson	991 руб.	Нет	2023-04-10	2026-01-24	Breakfast
8	Eric	Brown	448 руб.	Нет	2020-08-04	2024-04-05	Breakfast
9	Jim	Jackson	448 руб.	Нет	2020-08-04	2024-04-05	Breakfast
10	Eric	Smith	101 руб.	Нет	2024-01-23	2024-06-08	
11	Jim	Brown	111 руб.	Да	2025-02-01	2025-02-01	Breakfast

Подтверждение

Вы действительно хотите удалить бронирование #2 (Eric Smith)?

YesNo

Бронирование #2 успешно обновлено.

5. ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

5.1 Что такое REST API?

REST API — это программный интерфейс, построенный по архитектурному стилю REST. Он использует стандартные HTTP-запросы для работы с ресурсами: получения, создания, изменения и удаления данных. В REST клиент и сервер разделены, а каждый запрос содержит всю необходимую информацию для обработки.

5.2 Какие методы HTTP используются для CRUD-операций?

Для CRUD-операций обычно используются следующие HTTP-методы:

1. POST — создание ресурса.
2. GET — получение ресурса или списка ресурсов.
3. PUT или PATCH — обновление ресурса.
4. DELETE — удаление ресурса.

В разработанном приложении эти методы соответствуют операциям создания, просмотра, изменения и удаления бронирований.

5.3 Зачем нужен заголовок Accept?

Заголовок Accept сообщает серверу, в каком формате клиент ожидает получить ответ. Например, значение `application/json` означает, что клиент готов обрабатывать JSON. Это особенно важно при работе с REST API, где один и тот же ресурс теоретически может быть представлен в разных форматах.

5.4 Что такое JSON и зачем он нужен?

JSON — это текстовый формат обмена структурированными данными. Он удобен для передачи объектов, массивов, строк, чисел и логических значений между клиентом и сервером. В лабораторной работе JSON используется для отправки данных авторизации и бронирований, а также для чтения ответов REST API.

5.5 Как в C# десериализовать JSON в объект?

В C# JSON можно десериализовать с помощью библиотеки `Newtonsoft.Json`. Для этого используется метод `JsonConvert.DeserializeObject<T>()`, где `T` — тип объекта, в который нужно преобразовать строку JSON.

Пример:

```
string json = "{\"firstname\":\"Ivan\",\"lastname\":\"Ivanov\"}";  
Booking? booking = JsonConvert.DeserializeObject<Booking>(json);
```

5.6 Что такое асинхронное программирование и зачем оно нужно при работе с сетью?

Асинхронное программирование позволяет выполнять длительные операции без блокировки основного потока. При работе с сетью ответ сервера может приходиться не сразу, поэтому синхронный запрос привел бы к зависанию интерфейса. Использование `async` и `await` освобождает UI-поток на время ожидания ответа и возвращает выполнение после завершения операции.

5.7 Почему операции чтения не требуют токена, а операции изменения требуют?

Операции чтения `GET /booking` и `GET /booking/{id}` только получают открытые данные сервиса и не изменяют состояние на сервере, поэтому API разрешает выполнять их без авторизации. Операции изменения `PUT` и `DELETE` могут изменить или удалить ресурс, поэтому сервер требует подтверждение прав пользователя через токен авторизации.

5.8 Что произойдет, если отправить DELETE без заголовка Cookie?

Если отправить `DELETE /booking/{id}` без заголовка `Cookie: token=...`, сервер не сможет подтвердить права клиента на удаление записи. В результате

запрос будет отклонен кодом ошибки авторизации, а бронирование не будет удалено.

5.9 Что такое `Task.WhenAll()` и зачем загружать бронирования параллельно?

`Task.WhenAll()` ожидает завершения набора асинхронных задач. В лабораторной работе он используется для параллельной загрузки деталей бронирований по нескольким ID. Такой подход быстрее последовательной загрузки, потому что приложение не ждет завершения каждого сетевого запроса перед запуском следующего.

5.10 Почему `HttpClient` создается как `static readonly` и один на все приложение?

`HttpClient` хранит сетевые ресурсы и внутренние соединения. Если создавать новый экземпляр на каждый запрос, можно получить лишние накладные расходы и проблемы с исчерпанием сокетов. Поэтому в приложении используется один общий статический экземпляр `HttpClient`.

6. ВЫВОД

В ходе выполнения лабораторной работы было разработано приложение-клиент для взаимодействия с REST API сервиса `restful-booker.herokuapp.com`. Приложение реализует авторизацию, получение списка бронирований, загрузку подробной информации и полный набор CRUD-операций: создание, чтение, обновление и удаление бронирований.

Для сетевого взаимодействия использован `HttpClient`, а все HTTP-запросы выполняются асинхронно с применением `async/await`. Это позволяет не блокировать интерфейс Avalonia во время ожидания ответа сервера. Для преобразования данных между объектами C# и JSON используется библиотека `Newtonsoft.Json`.

Дополнительное задание выполнено: реализованы методы POST, PUT и DELETE, а для защищенных операций используется токен авторизации, полученный через POST /auth. Разработанное приложение демонстрирует основные принципы работы с HTTP, REST API, JSON и асинхронным программированием в C#.